



GIOLIT PROGRAM VERIFIER 7.0

Proving C programs correct

A practical tutorial: your first proof in five minutes, then the techniques, the results and the certificates behind it

Who this is for

Software developers, safety and design engineers, reviewers and researchers who want to know that C code *cannot* fail — or see the exact input that makes it fail.

What you need

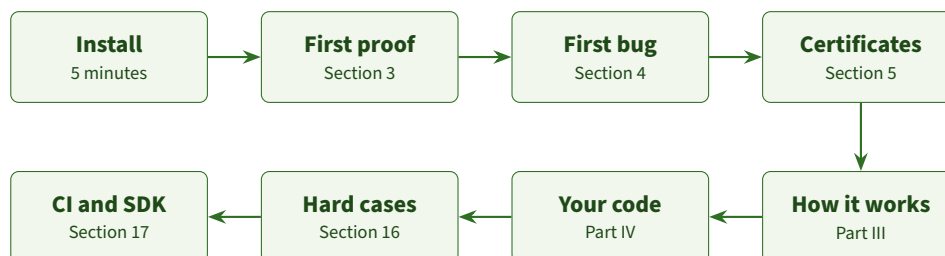
The verifier installed (Section 2), a terminal, and twenty minutes. No background in formal methods is assumed.

What you will learn

- prove that a program has no run-time errors, for every input;
- read a counterexample and turn it into a test;
- read certificates and hand them to clients and auditors;
- what the verifier assumes, and how it builds its proofs;
- what to do when a result is **UNKNOWN**;
- run it in continuous integration.

How to use this guide

Every program in this tutorial ships with the verifier, in `share/giolit/examples/` of your installation, and every output shown is the verifier's real output on that program — the same results are in the package's `results/tutorial-examples/` folder, with their certificates. Type the commands as you read.



In a hurry? Read Part I (Sections 1–5): it takes you from an empty terminal to a proof, a bug and a certificate.

Evaluating the product? Part I, then Section 14 (the example gallery) and Appendix D (benchmark results).

Verifying your own code? Part IV: which options suit which kind of program, and what to do when a result stays open.

Curious about the mathematics? Part III explains the proofs — Hoare logic, contracts and Owicki–Gries — with the examples.

 **Conventions** Commands you type are shown on a dark background, followed by the verifier's output. **PROVEN**, **VIO-LATED**, **UNREACHABLE** and **UNKNOWN** are the four possible results of a check. `v7` is the verifier's command.

Contents

How to use this guide	2
1 Why prove, when you can test?	4
2 Install in five minutes	5
3 Your first proof	5
4 Your first bug	6
5 Certificates and reports	7
6 What is checked, and what is assumed	9
6.1 Check kinds	9
6.2 Inputs and assumptions	9
7 Reading results	10
7.1 The console	10
7.2 Reading a certificate's proof	10
7.3 Reading a counterexample report	10
8 The result bundle, file by file	10
9 The verification model	12
10 Loops: inductive invariants	12

11 Functions: requires and ensures	13
11.1 Stating a contract yourself	13
11.2 Unit-level verification with --entry	14
12 Threads: Owicki-Gries	15
13 How proofs and counterexamples are found	16
14 Example gallery	17
15 Configurations by program type	19
16 When the result is UNKNOWN	19
17 Continuous integration and the SDK	19
18 Embedding the verifier: the SDK	20
18.1 What the SDK contains	20
18.2 Building a client	20
18.3 Example: verifying a C program	20
18.4 Example: verifying functions one by one	22
18.5 The other examples	23
19 Limits and good practice	24
A Command reference	25
B Trial and professional license	25
C Glossary	25
D Benchmark results	26

PART I

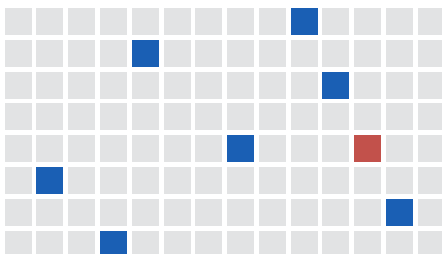
Getting started

From an empty terminal to a proof, a bug and a certificate: the verifier's value in five sections.

1 Why prove, when you can test?

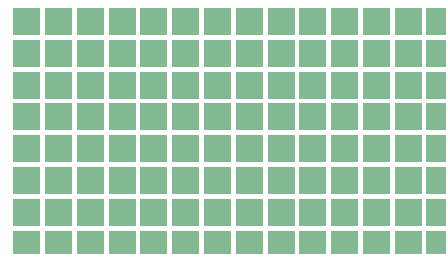
A test runs your program on a few inputs and checks the outcome. It shows that those inputs work. A *proof* covers *all* of them: every input, every path, every interleaving of threads. The Giolit Program Verifier proves — mathematically, and automatically — that a C program cannot fail an assertion, overflow an integer, divide by zero, index outside an array or misuse memory. When it cannot be proven because it is not true, the verifier gives you the input that breaks it.

Testing: 7 inputs checked



missed bug

Proof: every input covered



✓ PROVEN — or the failing input, if one exists

This matters most where failures are expensive: embedded controllers, medical and automotive software, protocol parsers, security boundaries, libraries used by thousands of callers. Standards such as DO-178C, ISO 26262 and IEC 61508 recognise formal proof as evidence; the verifier produces that evidence as a signed-off document, a *certificate*, together with everything a reviewer needs to check it.

The four results

Every potential error in the program becomes a *check*, and every check ends in one of four states:

Result	Meaning	What you do
PROVEN	No execution, for any input, violates it.	Nothing — the proof is in the certificate.
VIOLATED	A real execution violates it; you get the input and every step.	Fix the code; keep the input as a test.
UNREACHABLE	The operation never runs (dead code, or not called from the entry).	Check that this is intended.
UNKNOWN	Neither a proof nor a bug was found within the time budget.	Section 16 — usually more time, or a smaller scope.

 **Two guarantees PROVEN** is never wrong: the verifier over-approximates what a program can do, so a proof on its model is a proof for the program. **VIOLATED** is never a false alarm: every counterexample is replayed on the exact semantics of C before it is reported.

2 Install in five minutes

The verifier runs on 64-bit Linux (Ubuntu 24.04 or newer). Unpack the package and run the installer from its code folder:

```
$ tar xzf giolit-verifier-7.0.0-trial.tar.gz
$ cd giolit-verifier-7.0.0-trial/code
$ ./install.sh
```

The installer copies the programs to `~/local/giolit-v7`, puts the `v7` command on your PATH, checks that the system libraries are present (it prints the exact apt command if one is missing) and runs a first proof as a self-test. For PDF certificates, install LuaLaTeX once:

```
$ sudo apt install texlive-luatex texlive-latex-recommended texlive-latex-extra \
    texlive-pictures texlive-fonts-extra qpdf
```

The trial allows **1000 runs within 50 days**. `v7 --license` shows how many are left; it does not use a run. The README in `documents/` has the details, including installing for all users (`sudo ./install.sh --system`).

3 Your first proof

Go to the examples and look at the first program. It reads an input n , adds 2 to s exactly n times, and asserts that s ends up as $2*n$:

```
$ cd ~/local/giolit-v7/share/giolit/examples
```

```
1  /* Example 1 -- a loop and its invariant (Hoare logic).
2   * Expected: SAFE. The certificate's proof shows the loop invariant
3   * s = 2*i, 0 <= i <= n at the loop head; with i = n at the exit it gives
4   * s = 2*n, so the assertion holds for every input n. */
5  extern int __VERIFIER_nondet_int(void);
6  extern void __VERIFIER_assert(int);
7
8  int main(void) {
9      int n = __VERIFIER_nondet_int();
10     if (n < 0 || n > 1000) return 0;
11     int i = 0, s = 0;
12     while (i < n) {
13         s = s + 2;
14         i = i + 1;
15     }
16     __VERIFIER_assert(s == 2 * n);
17     return 0;
18 }
```

`__VERIFIER_nondet_int()` is an *input*: it can return any int. The if restricts it to $0 \leq n \leq 1000$. Now verify:

```
$ v7 01_loop_invariant.c
v7 7.0.0 -- entry main, 6 checks

01_loop_invariant.c:13:11: PROVEN [overflow] int32 overflow in + (in main)
01_loop_invariant.c:14:11: PROVEN [overflow] int32 overflow in + (in main)
01_loop_invariant.c:16:3: PROVEN [assertion] __VERIFIER_assert condition false (in main)
01_loop_invariant.c:16:28: PROVEN [overflow] int32 overflow in * (in main)
01_loop_invariant.c:18:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in main)
01_loop_invariant.c:18:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in main)

6 checks: 6 proven, 0 unreachable, 0 violated, 0 unknown (0.7 s)
```

Result: ALL CHECKS PROVEN

The verifier created a check for the assertion and one for every operation that could fail at run time — here the additions and the multiplication, which could overflow a 32-bit int. It *proved* all of them, for every one of the 1001 allowed values of n and every path through the loop. A unit test would try a handful of values; this proof covers them all, and it would cover all 2^{32} values of an unrestricted input just the same.

Tip Each line names the file, line and column of the check, its kind in brackets, and what it guards against. The last two lines summarise the run; the exit code (0 here) summarises it for scripts.

4 Your first bug

The second example is the same program with one extra line in the loop — the kind of special case that slips into real code:

```
1  /* Example 2 -- the same loop with a bug in iteration 7.
2   * Expected: UNSAFE. The report gives a failing input (n = 8 or larger) and
3   * the execution step by step: s gets an extra 1 when i == 7, so s != 2*n. */
4  extern int __VERIFIER_nondet_int(void);
5  extern void __VERIFIER_assert(int);
6
7  int main(void) {
8      int n = __VERIFIER_nondet_int();
9      if (n < 0 || n > 1000) return 0;
10     int i = 0, s = 0;
11     while (i < n) {
12         s = s + 2;
13         if (i == 7) s = s + 1; /* the bug */
14         i = i + 1;
15     }
16     __VERIFIER_assert(s == 2 * n);
17     return 0;
18 }
```

```
$ v7 02_loop_bug.c
v7 7.0.0 -- entry main, 7 checks

02_loop_bug.c:12:11: PROVEN [overflow] int32 overflow in + (in main)
02_loop_bug.c:13:23: PROVEN [overflow] int32 overflow in + (in main)
02_loop_bug.c:14:11: PROVEN [overflow] int32 overflow in + (in main)
02_loop_bug.c:16:3: VIOLATED [assertion] __VERIFIER_assert condition false (in main)
... 3 earlier steps
02_loop_bug.c:12:11 n=8, i=0, s=0
02_loop_bug.c:12:11 n=8, i=1, s=2
02_loop_bug.c:12:11 n=8, i=2, s=4
02_loop_bug.c:12:11 n=8, i=3, s=6
02_loop_bug.c:12:11 n=8, i=4, s=8
02_loop_bug.c:12:11 n=8, i=5, s=10
02_loop_bug.c:12:11 n=8, i=6, s=12
02_loop_bug.c:12:11 n=8, i=7, s=14
02_loop_bug.c:13:23 n=8, i=7, s=16
02_loop_bug.c:16:28 n=8, s=17
02_loop_bug.c:16:23 s=17
02_loop_bug.c:16:3
02_loop_bug.c:16:3 <-- assertion violated here
02_loop_bug.c:16:28: PROVEN [overflow] int32 overflow in * (in main)
02_loop_bug.c:18:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in main)
02_loop_bug.c:18:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in main)

7 checks: 6 proven, 0 unreachable, 1 violated, 0 unknown (0 s)
Result: VIOLATIONS FOUND
```

The verifier found the input that breaks the assertion, $n = 8$, and the complete execution: each line is one step, with the values of the variables at that point. At iteration $i = 7$ the special case adds the extra 1, and the assertion at

the end sees $s = 17$ instead of 16. Only inputs $n \geq 8$ trigger the bug — a test suite that tried $n = 0, 1, 5$ would have passed.

Turn it into a test Replay the counterexample as an ordinary program: give the input function the failing value and compile with the sanitizers.

```
/* repro_02.c -- replays the counterexample of 02_loop_bug.c */
#include <assert.h>
int __VERIFIER_nondet_int(void) { return 8; } /* the failing input */
void __VERIFIER_assert(int c) { assert(c); }
```

cc -g -fsanitize=address,undefined repro_02.c 02_loop_bug.c && ./a.out stops at the assertion. Keep it as a regression test.

5 Certificates and reports

Add `--report=DIR` to any command and the verifier writes a self-contained folder: a PDF for people, and the evidence behind it.

```
$ v7 --report=out/01 01_loop_invariant.c # out/01/certificate.pdf
$ v7 --report=out/02 02_loop_bug.c # out/02/counterexample-report.pdf
```

Certificate of Formal Verification — Trial version GIO-2CB4963F1D6

GIOLIT | FORMAL VERIFICATION OF C PROGRAMS
Certificate of Formal Verification
Document GIO-2CB4963F1D6 (issued 2020-10-01 18:03:44 +0530) (UTC+0530)

TRIAL VERSION. This document was generated by the trial version of the Giolit Program Verifier, for evaluation. The verification results are real; the document is not a signed certificate.

01_loop_invariant.c is free of the errors listed below.
For every possible input, no execution of 01_loop_invariant.c (starting at main()) can:

- fail an assertion (1 place checked)
- overflow a signed integer (2 places checked)
- lose allocated memory (memory leak) (1 place checked)
- exit with memory still allocated (1 place checked)

This is a mathematical proof covering all executions, not a test of some of them. The proof itself is part of this certificate (page 4).

Program file
Location: SHA-256
Size: 33 lines, entry function main
Machine ID: Fw954868F5418681d61e1c7b9e9
Machine: glibc build — Ubuntu 24.04.5 LTS (kernel 6.8.0-142-generic); Intel(R) Core(TM) i5-8350U CPU @ 1.70GHz
Verified by: glibc/glibc build
Engine: Giolit Program Verifier v7.0.0, build e6a236-modified (trial version)
Date and time: 2020-10-01 18:03:44 +0530 (UTC+0530) (analysis 0.7 s)
Technique: Inductive invariants (abstract interpretation), Concrete execution, Decided statically
Edition: Trial version (for evaluation; documents are not signed)

About this trial document

- Same file.** sha256sum 01_loop_invariant.c; must print 6274851acaf27a5a7a7b71f82b09f4a7a2f7a7c2d4984ba79a77f.
- Same result.** reproduce.sh re-runs the verification.
- For evaluation.** This document shows what the Giolit Program Verifier proves and finds. It was produced by the trial version and is not signed; certificates your clients can check independently are issued by the full version.

GIOLIT — Formal Verification Services
Mathematical proof that your software is free of run-time errors.
We offer formal verification as a service: proofs of absence of run-time errors (overflow, division by zero, out-of-bounds access, invalid pointers, memory leaks) and of functional properties for C code, certificates for safety standards (DO-178C, ISO 26262, IEC 61508), and review of counterexamples with your engineers.
www.giolit.com business.giolitlabs@gmail.com

www.giolit.com business.giolitlabs@gmail.com Formal verification services Page 1 of 5

Certificate — all checks proven: the verdict in plain words, what was verified (file, SHA-256 hash, machine, engine, date) and how to check it.

Verification Report: Errors Found — Trial version GIO-75F9C4B77478

GIOLIT | FORMAL VERIFICATION OF C PROGRAMS
Formal Verification Report: Errors Found
Document GIO-75F9C4B77478 (issued 2020-10-01 18:03:51 +0530) (UTC+0530)

TRIAL VERSION. This document was generated by the trial version of the Giolit Program Verifier, for evaluation. The verification results are real; the document is not a signed certificate.

02_loop_bug.c contains an error.
A concrete execution was found in which:
✗ **line 16:** the assertion fails — with input __VERIFIER_nondet_int()=8
Proven for every input: it can never

- overflow a signed integer (1 place checked)
- lose allocated memory (memory leak) (1 place checked)
- exit with memory still allocated (1 place checked)

The failing execution is shown step by step on page 4 and can be replayed.

Program file
Location: SHA-256
Size: 33 lines, entry function main
Machine ID: Fw954868F5418681d61e1c7b9e9
Machine: glibc/glibc build
Engine: Giolit Program Verifier v7.0.0, build e6a236-modified (trial version)
Date and time: 2020-10-01 18:03:51 +0530 (UTC+0530) (analysis 0.6 s)
Technique: Inductive invariants (abstract interpretation), Concrete execution, Decided statically
Edition: Trial version (for evaluation; documents are not signed)

About this trial document

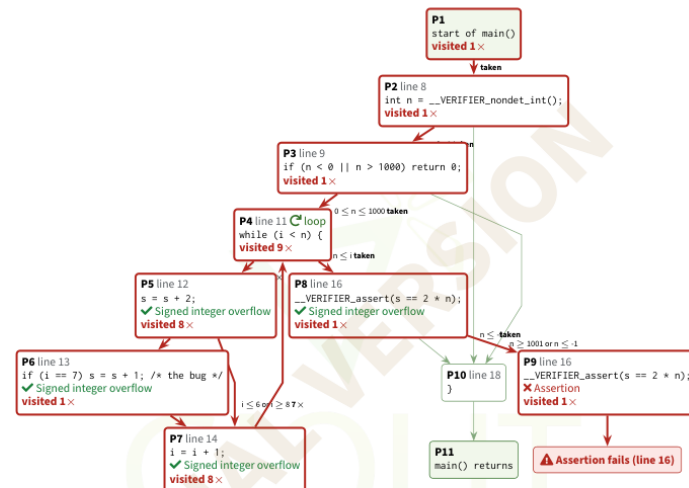
- Same file.** sha256sum 02_loop_bug.c; must print 75f5178c3c4a4d533f8ba3c278a9d7f8f5a38a21a713771f1f093238.
- Same result.** reproduce.sh re-runs the verification.
- For evaluation.** This document shows what the Giolit Program Verifier proves and finds. It was produced by the trial version and is not signed; certificates your clients can check independently are issued by the full version.

GIOLIT — Formal Verification Services
Mathematical proof that your software is free of run-time errors.
We offer formal verification as a service: proofs of absence of run-time errors (overflow, division by zero, out-of-bounds access, invalid pointers, memory leaks) and of functional properties for C code, certificates for safety standards (DO-178C, ISO 26262, IEC 61508), and review of counterexamples with your engineers.
www.giolit.com business.giolitlabs@gmail.com

www.giolit.com business.giolitlabs@gmail.com Formal verification services Page 1 of 5

Counterexample report — an error was found: where, the failing input, and what was still proven.

The certificate continues with the program, one row per check with the method that decided it, and the *proof*: the program's control-flow graph with the invariant that holds at every point (Section 7). The counterexample report draws the failing execution on the same graph and lists every step:

Counterexample 1: Assertion at line 16**Check:** `__VERIFIER_assert` condition false (in `main()`, CWE-617).**Found by:** Concrete execution.**Failing input:** `__VERIFIER_nondet_int()`=8**The failing execution on the control-flow graph.** Red arrows are the transfers this execution takes (with how often); red boxes are the points it visits. The table after the graph lists every step with the values of the variables when execution arrives at the point (changed values in bold).**Function `main()`****The walk, step by step**

Step	Point	Line	Source	Variable values on arrival
1	P1		start of <code>main()</code>	
2	P2	8	<code>int n = __VERIFIER_nondet_int();</code>	
3	P3	9	<code>if (n < 0 n > 1000) return 0;</code>	n = 8
4	P4	11	<code>while (i < n) {</code>	n = 8, i = 0, s = 0
5	P5	12	<code>s = s + 2;</code>	n = 8, i = 0, s = 0
6	P7	14	<code>i = i + 1;</code>	n = 8, i = 0, s = 2
7	P4	11	<code>while (i < n) {</code>	n = 8, i = 1, s = 2
8	P5	12	<code>s = s + 2;</code>	n = 8, i = 1, s = 2
9	P7	14	<code>i = i + 1;</code>	n = 8, i = 1, s = 4
10	P4	11	<code>while (i < n) {</code>	n = 8, i = 2, s = 4
11	P5	12	<code>s = s + 2;</code>	n = 8, i = 2, s = 4
12	P7	14	<code>i = i + 1;</code>	n = 8, i = 2, s = 6
13	P4	11	<code>while (i < n) {</code>	n = 8, i = 3, s = 6
14	P5	12	<code>s = s + 2;</code>	n = 8, i = 3, s = 6
15	P7	14	<code>i = i + 1;</code>	n = 8, i = 3, s = 8
16	P4	11	<code>while (i < n) {</code>	n = 8, i = 4, s = 8
17	P5	12	<code>s = s + 2;</code>	n = 8, i = 4, s = 8

⚠ Trial documents Documents made by the trial are marked “Trial version” on every page. With a purchased license they are professional documents: your license number and company name at the top, no trial marks, and each PDF protected by its own password (delivered in `certificate-password.txt` next to it; Appendix B).

Understanding the results

What is checked, what is assumed, and how to read every result, certificate and file the verifier produces.

6 What is checked, and what is assumed

6.1 Check kinds

Kind	A violation is	CWE
assertion	<code>assert(c)</code> or <code>__VERIFIER_assert(c)</code> with <code>c</code> false, or <code>reach_error()</code> reached	CWE-617
overflow	<code>signed + - * <<</code> , or <code>INT_MIN / -1</code> , leaving the type's range (undefined behaviour)	CWE-190
div-by-zero	<code>/</code> or <code>%</code> by zero	CWE-369
shift	a shift amount that is negative or at least the operand's width	CWE-1335
array-bounds	an index outside a declared array	CWE-787
valid-deref	a read or write through a null, dangling or out-of-bounds pointer	CWE-476, 416
valid-free	a double free, or free of memory not from <code>malloc</code>	CWE-415, 761
valid-memtrack	allocated memory that is no longer reachable (a leak)	CWE-401
valid-memcleanup	memory still allocated when the program exits	CWE-401

Choose them with `--checks=` (default all). Groups: arithmetic (overflow, div-by-zero, shift), memsafety (valid-deref, valid-free, valid-memtrack) and rte (everything but assertions). Checking only what matters makes hard programs faster (Section 16).

6.2 Inputs and assumptions

A result holds for every execution that starts at the entry function (`main`, or the one named by `--entry`), where:

- every **input** can have any value of its type. Inputs are the results of functions whose body is not in the analysed files (such as `__VERIFIER_nondet_int()` or a driver's `read_sensor()`), the entry function's parameters, volatile variables (hardware registers) and data of the environment;
- `__VERIFIER_assume(c)` or `assume_abort_if_not(c)` restricts the inputs: executions where `c` is false are not considered. Use it to state what the environment guarantees (a sensor reads 0–4095);
- the machine is the host's C data model (x86-64: `int` 32 bits, `long` and pointers 64 bits) with two's-complement integers.

Operations the model cannot represent exactly — floating-point arithmetic, some bit manipulations — are treated as “any value of the result type”. That can prevent a proof (**UNKNOWN**); it never produces a false **PROVEN**.

7 Reading results

7.1 The console

One line per check (file:line:column: RESULT [kind] message (in function)), a counterexample under each **VIOLATED** check, a summary line and a Result: line. An **UNKNOWN** check shows why it stayed open:

Reason	Meaning
WALL_CLOCK	the time budget ran out
NO_CONFLICT_UNPROVEN	no counterexample exists in the explored model, but the invariants found were not strong enough for a proof
CONFLICT_UNRESOLVED	a potential counterexample could neither be confirmed nor refuted
INVARIANTS_FAILED	the invariant computation did not converge
ITERATION_CAP, FORWARD_CAP	an internal size limit was reached (a very large state space)

7.2 Reading a certificate's proof

For each function the certificate draws the control-flow graph. Each box is a program point P1, P2, ... with its line and source; the green text is the *annotation* — a fact that holds every time execution reaches that point. For Example 1:

Point	Line	Source	Annotation
P3	10	if (n < 0 n > 1000) return 0;	$n \geq 0$
P4	12	while (i < n) {	$s = 2i \wedge n \leq 1000 \wedge s \geq 0 \wedge i \leq n$
P5	13-14	s = s + 2; i = i + 1;	$s = 2i \wedge n \leq 1000 \wedge s \geq 0 \wedge i < n$
P6	16	__VERIFIER_assert(s == 2 * n);	$s = 2n \wedge 0 \leq s \leq 2000$
P7	16	the assertion's failure branch	false — never reached

You can check the proof by hand: each annotation follows from the annotations before it and the code in between, and the failure branch's annotation is false. That is all a proof is (Section 10).

7.3 Reading a counterexample report

The failing execution is drawn in red on the same graph: red arrows are the transfers it takes (with how often, e.g. a loop body taken 8×), red boxes the points it visits. “The walk, step by step” lists every step with the variables' values; changed values are bold, so the moment things go wrong stands out.

8 The result bundle, file by file

File	For whom, and what for
certificate.pdf, counterexample-report.pdf, verification-report.pdf	people — managers, reviewers, customers, auditors: the result in plain words, the proof or the failing execution
program.c (and its local headers)	reviewers: exactly the code that was verified (its SHA-256 is on the cover)
results.json	scripts and dashboards: one entry per check, with traces for violations
results.sarif	GitHub code scanning, GitLab, IDEs: violations and open checks as code annotations (SARIF 2.1.0, CWE-tagged)
proof.json	tool builders and researchers: the complete model and every invariant, for independent re-checking
certificate.json	audit trails: verdict, file hash, machine, engine, date, every check
verifier-output.txt, reproduce.sh	everyone: the console output, and the command that re-runs the verification
certificate-password.txt	licensed documents only: the PDF's password

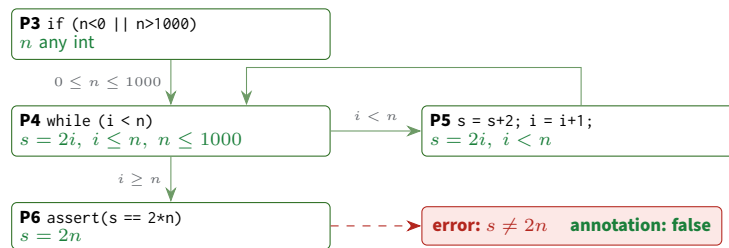
💡 **Evidence you can hand over** The bundle is designed to leave your hands: attach it to a release, a pull request or a safety case. `reproduce.sh` lets a reviewer re-run the verification on their own machine.

How it works

The verification model, and the three proof principles the verifier automates: inductive invariants, function contracts, and Owicki–Gries reasoning for threads.

9 The verification model

The program is compiled by Clang 18 and translated into a *control-flow graph* whose transitions are exact relations over the program's variables: machine-integer ranges and wrap-around, pointers as object and offset, and the heap's allocation state are all modelled. Every check becomes an *error edge*, taken exactly when the check fails. Verifying a check means deciding whether any execution can take its error edge.



In logical terms the graph is a set of *constrained Horn clauses*: an unknown predicate Inv_P per point P , one clause per transition, for example $Inv_{P4}(i, s, n) \wedge i < n \rightarrow Inv_{P5}(i, s, n)$. A **proof** is a solution — a formula for every Inv_P that satisfies all clauses and makes every error edge impossible; those formulas are the annotations printed in the certificate. A **counterexample** is a chain of clauses from the start to an error edge, with concrete values.

10 Loops: inductive invariants

A Hoare triple $\{P\} C \{Q\}$ says: if P holds before C runs, Q holds after it. Loops are handled by the rule at the heart of every proof:

$$\frac{\{I \wedge b\} \text{body} \{I\}}{\{I\} \text{while}(b) \text{body} \{I \wedge \neg b\}}$$

I is the *loop invariant*. For Example 1 the verifier found $I \equiv s = 2i \wedge i \leq n \wedge n \leq 1000 \wedge s \geq 0$, and three facts make it a proof — all three checked automatically:

Initiation	On entry $i = 0, s = 0$ and $0 \leq n \leq 1000$, so I holds.
Consecution	If I holds and $i < n$, after $s = s + 2; i = i + 1$; it holds again: $s + 2 = 2(i + 1)$.
Safety	At the exit, $I \wedge i \geq n$ gives $i = n$, hence $s = 2n$: the assertion cannot fail.

Key point You never write the invariant: the verifier discovers it. It computes, for every point, the strongest facts it can express — intervals, differences $x - y \leq c$ and relations such as $s - 2i$ found in the program itself — until they stop changing.

11 Functions: requires and ensures

For each function the verifier computes a *contract*: what holds at every call (**requires**, over the parameters) and at every return (**ensures**, relating the parameters and the result). A call is then verified against the contract instead of re-analysing the body — this is what makes recursion tractable. Example 3:

```
1  /* Example 3 -- a recursive function: requires/ensures (function summaries).
2   * Expected: SAFE. The verifier infers a contract for triangle():
3   *   requires  0 <= n <= 1000      (every call satisfies it)
4   *   ensures   result >= n        (every return satisfies it)
5   * and proves main's assertion and every addition (no overflow) from it. */
6  extern int __VERIFIER_nondet_int(void);
7  extern void __VERIFIER_assert(int);
8
9  int triangle(int n) {
10     if (n == 0) return 0;
11     return n + triangle(n - 1);
12 }
13
14 int main(void) {
15     int n = __VERIFIER_nondet_int();
16     if (n < 0 || n > 1000) return 0;
17     int t = triangle(n);
18     __VERIFIER_assert(t >= n);
19     return 0;
20 }
```

```
$ v7 03_contracts.c
v7 7.0.0 -- entry main, 5 checks

03_contracts.c:11:12: PROVEN [overflow] int32 overflow in + (in triangle)
03_contracts.c:11:25: PROVEN [overflow] int32 overflow in - (in triangle)
03_contracts.c:18:3: PROVEN [assertion] __VERIFIER_assert condition false (in main)
03_contracts.c:20:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in main)
03_contracts.c:20:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in main)

5 checks: 5 proven, 0 unreachable, 0 violated, 0 unknown (1.4 s)
Result: ALL CHECKS PROVEN
```

The certificate shows the contract of `triangle()`: requires $0 \leq n \leq 1000$, ensures $n \leq \text{result}$. It is proven by induction: for $n = 0$ the result is 0; otherwise the inner call satisfies the precondition, so $\text{triangle}(n-1) \geq n-1 \geq 0$ and the result $n + \text{triangle}(n-1) \geq n$. In main, the contract gives $t \geq n$ — the assertion.

11.1 Stating a contract yourself

To prove that a function meets a specification *you* choose, write a small harness and verify it as the entry point (Example 12):

```
1  void mid_contract(void) {
2     int lo = __VERIFIER_nondet_int(), hi = __VERIFIER_nondet_int();
3     __VERIFIER_assume(0 <= lo && lo <= hi); /* requires */
4     int m = mid(lo, hi);
5     __VERIFIER_assert(lo <= m && m <= hi); /* ensures */
6 }
```

```
$ v7 --entry=mid_contract 12_contract_harness.c
v7 7.0.0 -- entry mid_contract, 6 checks

12_contract_harness.c:13:13: PROVEN [overflow] int32 overflow in + (in mid)
12_contract_harness.c:13:19: PROVEN [overflow] int32 overflow in - (in mid)
```

```
12_contract_harness.c:20:3: PROVEN [assertion] __VERIFIER_assert condition false (in mid_contract)
12_contract_harness.c:21:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in mid_contract)
12_contract_harness.c:21:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in
mid_contract)
12_contract_harness.c:27:3: UNREACHABLE [assertion] __VERIFIER_assert condition false (in mid_contract_wrong)

6 checks: 5 proven, 1 unreachable, 0 violated, 0 unknown (0 s)
Result: ALL CHECKS PROVEN
```

A false contract is refuted with concrete values — the harness `mid_contract_wrong` claims $lo < m$:

```
$ v7 --entry=mid_contract_wrong 12_contract_harness.c
v7 7.0.0 -- entry mid_contract_wrong, 6 checks

12_contract_harness.c:13:13: PROVEN [overflow] int32 overflow in + (in mid)
12_contract_harness.c:13:19: PROVEN [overflow] int32 overflow in - (in mid)
12_contract_harness.c:20:3: UNREACHABLE [assertion] __VERIFIER_assert condition false (in mid_contract)
12_contract_harness.c:27:3: VIOLATED [assertion] __VERIFIER_assert condition false (in mid_contract_wrong)
12_contract_harness.c:24:12
12_contract_harness.c:24:12 lo=11
12_contract_harness.c:24:42 lo=11, hi=11
12_contract_harness.c:25:35 lo=11, hi=11
12_contract_harness.c:13:25 lo=11
12_contract_harness.c:27:24 lo=11, m=11
12_contract_harness.c:27:3 <-- assertion violated here
12_contract_harness.c:28:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in mid_contract_wrong)
12_contract_harness.c:28:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in
mid_contract_wrong)

6 checks: 4 proven, 1 unreachable, 1 violated, 0 unknown (0 s)
Result: VIOLATIONS FOUND
```

11.2 Unit-level verification with `--entry`

`--entry=f` verifies `f` for *all* possible arguments — a result for every caller, present and future (Example 11):

```
$ v7 --entry=mid_bad 11_entry_points.c
v7 7.0.0 -- entry mid_bad, 6 checks

11_entry_points.c:7:3: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in mid_bad)
11_entry_points.c:7:3: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in mid_bad)
11_entry_points.c:7:14: VIOLATED [overflow] int32 overflow in + (in mid_bad)
counterexample at node mid_bad:bb0(0,0,0,0,0,0,0,-2147483648,-2147483648,0,0,0,0) steps into @error (concrete
derivation, 62 tuples tabulated, 0 exact cycle accelerations)
validated backward chain (@entry -> ... -> conflict): mid_bad:entry(0,0,0,0,-2147483648,-2147483648) -> mid_bad:bb0
(0,0,0,0,0,0,0,-2147483648,-2147483648,0,0,0,0) -> @error()
validated forward chain (conflict -> ... -> @error): @error
11_entry_points.c:7:14 <-- overflow violated here
11_entry_points.c:12:13: UNREACHABLE [overflow] int32 overflow in + (in mid_good)
11_entry_points.c:12:19: UNREACHABLE [overflow] int32 overflow in - (in mid_good)
11_entry_points.c:15:41: UNREACHABLE [overflow] int32 overflow in - (in main)

6 checks: 2 proven, 3 unreachable, 1 violated, 0 unknown (0 s)
Result: VIOLATIONS FOUND
```

$(lo + hi) / 2$ overflows for large arguments — the famous binary search bug. $lo + (hi - lo) / 2$ is proven for every argument:

```
$ v7 --entry=mid_good 11_entry_points.c
v7 7.0.0 -- entry mid_good, 6 checks

11_entry_points.c:7:14: UNREACHABLE [overflow] int32 overflow in + (in mid_bad)
11_entry_points.c:12:13: PROVEN [overflow] int32 overflow in + (in mid_good)
11_entry_points.c:12:19: PROVEN [overflow] int32 overflow in - (in mid_good)
11_entry_points.c:13:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in mid_good)
11_entry_points.c:13:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in mid_good)
11_entry_points.c:15:41: UNREACHABLE [overflow] int32 overflow in - (in main)

6 checks: 4 proven, 2 unreachable, 0 violated, 0 unknown (0 s)
```

Result: ALL CHECKS PROVEN

12 Threads: Owicki–Gries

For multithreaded programs (POSIX threads, mutexes, read-write locks, condition variables, C11 atomics) the verifier builds an *Owicki–Gries* proof:

1. **a sequential proof per thread:** every point of every thread gets an annotation, as in Section 10;
2. **interference freedom:** every annotation must stay true when *any other thread* takes *any* step that can run concurrently with it. The verifier generates and proves one stability condition per pair (annotation, other thread's step).

With both, no interleaving — out of the astronomically many — can falsify an annotation. Example 6: two workers increment a shared counter under a mutex, and main checks $x == 2$ after joining both.

```

1  /* Example 6 -- two threads increment a shared counter under a mutex
2   * (Owicki-Gries). Expected: SAFE: after both joins, x == 2. */
3  #include <pthread.h>
4  extern void __assert_fail(const char *, const char *, unsigned int, const char *);
5  void reach_error(void) { __assert_fail("0", "06_threads_mutex.c", 5, "reach_error"); }
6
7  int x = 0;
8  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
9
10 void *worker(void *arg) {
11     pthread_mutex_lock(&m);
12     x = x + 1;
13     pthread_mutex_unlock(&m);
14     return 0;
15 }
16
17 int main(void) {
18     pthread_t a, b;
19     pthread_create(&a, 0, worker, 0);
20     pthread_create(&b, 0, worker, 0);
21     pthread_join(a, 0);
22     pthread_join(b, 0);
23     if (x != 2) reach_error();
24     return 0;
25 }
```

```

$ v7 06_threads_mutex.c
v7 7.0.0 -- entry main, 2 checks

06_threads_mutex.c:12:9: PROVEN [overflow] int32 overflow in + (in worker)
06_threads_mutex.c:23:15: PROVEN [assertion] assertion failure (reach_error reached) (in main)

2 checks: 2 proven, 0 unreachable, 0 violated, 0 unknown (0.3 s)
Result: ALL CHECKS PROVEN
```

Example 7 removes the mutex. A stability condition fails, and the verifier returns the interleaving that breaks the program — both workers read $x = 0$ before either writes:

```

$ v7 07_threads_race.c
v7 7.0.0 -- entry main, 2 checks

07_threads_race.c:11:9: PROVEN [overflow] int32 overflow in + (in worker)
07_threads_race.c:21:15: VIOLATED [assertion] assertion failure (reach_error reached) (in main)
07_threads_race.c:16:13  x=0
07_threads_race.c:17:3   x=0
07_threads_race.c:18:3   x=0
07_threads_race.c:11:7
07_threads_race.c:19:3   x=0
07_threads_race.c:11:5   x=1
07_threads_race.c:19:3   x=1
```

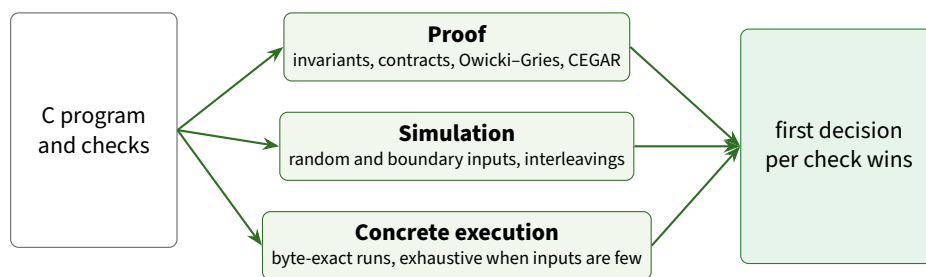
```
07_threads_race.c:20:3  x=1
07_threads_race.c:21:7
07_threads_race.c:21:15  <-- assertion violated here

2 checks: 1 proven, 0 unreachable, 1 violated, 0 unknown (0.1 s)
Result: VIOLATIONS FOUND
```

💡 **Lock-free protocols** Algorithms that synchronise through ordinary variables, such as Peterson’s mutual exclusion (Example 8), need annotations that depend on *where the other thread is*. The verifier adds these location annotations automatically when the plain proof is not enough; allow `--timeout=60` or more.

13 How proofs and counterexamples are found

The verifier runs a portfolio of techniques *in parallel*: a proof engine, a random interleaving of the threads (programs with threads) and a concrete execution of the program race each other in separate processes, and the first to decide a check settles it. Easy bugs are found in milliseconds by execution; proofs arrive as soon as the invariants are strong enough.



Technique	What it does
Abstract interpretation	invariants for every point over template polyhedra (intervals, octagons, relations discovered from the program), with widening and narrowing
Path-sensitive discharge	proves each path into a failing operation infeasible, with an exact Farkas certificate
Horn-clause solving with CEGAR	solves the whole program; each abstract counterexample is checked against the exact semantics and, if spurious, refines the abstraction
Disjunctive analysis	keeps a bounded union of invariants per point (one case per branch history or small value range) when one convex invariant is not enough; the certificate lists every case
Random interleavings and concrete execution	find bugs fast (interleavings: programs with threads); small input domains are enumerated exhaustively
Validated counterexamples	every failure is replayed step by step on the exact semantics before it is reported

PART IV

Using it on your code

The shipped examples at a glance, the right configuration for each kind of program, what to do when a result stays open, and continuous integration.

14 Example gallery

All twelve examples are in `share/giolit/examples/`; their results, certificates and reports are in the package's `results/tutorial-examples/`.

Heap memory, used correctly

04_heap_safe.c

A buffer is allocated, filled in bounds and freed. Every access, the free and the absence of leaks are proven.

```
$ v7 04_heap_safe.c
v7 7.0.0 -- entry main, 5 checks

04_heap_safe.c:12:27: PROVEN [overflow] int32 overflow in + (in main)
04_heap_safe.c:12:36: PROVEN [valid-deref] invalid memory access (null, dangling or out of bounds) (in main)
04_heap_safe.c:13:3: PROVEN [valid-free] invalid free (not a live heap object, or not its start) (in main)
04_heap_safe.c:15:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in main)
04_heap_safe.c:15:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in main)

5 checks: 5 proven, 0 unreachable, 0 violated, 0 unknown (0.8 s)
Result: ALL CHECKS PROVEN
Note: 1 operations were over-approximated (e.g. unmodelled memory); violations are only reported along exactly
modelled paths.
```

An off-by-one write

05_heap_bug.c

The loop writes one element past the end of the buffer. The counterexample shows the index and the buffer's size at the failing write.

```
$ v7 05_heap_bug.c
v7 7.0.0 -- entry main, 5 checks

05_heap_bug.c:11:28: PROVEN [overflow] int32 overflow in + (in main)
05_heap_bug.c:11:37: VIOLATED [valid-deref] invalid memory access (null, dangling or out of bounds) (in main)
... 993 earlier steps
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=988
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=989
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=990
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=991
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=992
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=993
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=994
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=995
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=996
05_heap_bug.c:11 n=999, a.tag=1, a.off=0, i=997
```

```
05_heap_bug.c:11: n=999, a.tag=1, a.off=0, i=998
05_heap_bug.c:11: n=999, a.tag=1, a.off=0, i=999
05_heap_bug.c:11:37 <-- valid-deref violated here
05_heap_bug.c:12:3: PROVEN [valid-free] invalid free (not a live heap object, or not its start) (in main)
05_heap_bug.c:14:1: PROVEN [valid-memcleanup] allocated memory not freed at program exit (in main)
05_heap_bug.c:14:1: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (in main)

5 checks: 4 proven, 0 unreachable, 1 violated, 0 unknown (0.1 s)
Result: VIOLATIONS FOUND
Note: 1 operations were over-approximated (e.g. unmodelled memory); violations are only reported along exactly
modelled paths.
```

Peterson's mutual exclusion

08_peterson.c

A lock-free protocol: the critical sections never overlap. A hard case, proven with location annotations (Section 12).

```
$ v7 08_peterson.c
v7 7.0.0 -- entry main, 2 checks

08_peterson.c:19:18: PROVEN [assertion] assertion failure (reach_error reached) (in thr1)
08_peterson.c:29:18: PROVEN [assertion] assertion failure (reach_error reached) (in thr2)

2 checks: 2 proven, 0 unreachable, 0 violated, 0 unknown (5.6 s)
Result: ALL CHECKS PROVEN
```

An industrial motor controller

09_motor_controller.c

A state machine in the style of IEC 61508: every transition, counter and table access proven safe. No heap, so the memory checks are left out.

```
$ v7 --checks=assertion,arithmetic,array-bounds 09_motor_controller.c
v7 7.0.0 -- entry main, 3 checks

09_motor_controller.c:20:18: PROVEN [overflow] int32 overflow in + (in step)
09_motor_controller.c:43:9: PROVEN [assertion] assertion failure (reach_error reached) (in main)
09_motor_controller.c:44:9: PROVEN [assertion] assertion failure (reach_error reached) (in main)

3 checks: 3 proven, 0 unreachable, 0 violated, 0 unknown (0.2 s)
Result: ALL CHECKS PROVEN
```

Avionics: median voting

10_sensor_voting.c

Median voting over three redundant sensors, with fault handling — proven for every combination of readings.

```
$ v7 --checks=assertion,arithmetic,array-bounds 10_sensor_voting.c
v7 7.0.0 -- entry main, 9 checks

10_sensor_voting.c:23:28: PROVEN [overflow] int32 overflow in - (in monitor)
10_sensor_voting.c:25:13: PROVEN [array-bounds] memory access out of bounds (in monitor)
10_sensor_voting.c:25:37: PROVEN [array-bounds] memory access out of bounds (in monitor)
10_sensor_voting.c:26:13: PROVEN [array-bounds] memory access out of bounds (in monitor)
10_sensor_voting.c:26:40: PROVEN [array-bounds] memory access out of bounds (in monitor)
10_sensor_voting.c:28:19: PROVEN [array-bounds] memory access out of bounds (in monitor)
10_sensor_voting.c:36:9: PROVEN [assertion] assertion failure (reach_error reached) (in main)
10_sensor_voting.c:44:9: PROVEN [assertion] assertion failure (reach_error reached) (in main)
10_sensor_voting.c:44:9: PROVEN [array-bounds] memory access out of bounds (in main)

9 checks: 9 proven, 0 unreachable, 0 violated, 0 unknown (1.7 s)
Result: ALL CHECKS PROVEN
```

15 Configurations by program type

Program	Configuration	Notes
Arithmetic and loops	v7 file.c (all checks, 60 s)	most loops are decided in seconds
Embedded / industrial control	--checks=assertion,arithmetic,array-bounds	no heap: leaving out the memory checks saves time (Examples 9, 10)
Heap-manipulating code	--checks=memsafety,valid-memcleanup	add assertion as needed; allow 30–60 s
Recursive functions	default	contracts per function (Section 11)
Library or API functions	--entry=function	arbitrary arguments: valid for every caller (Example 11)
Against your specification	a harness, --entry=harness	Example 12
Threads with locks	default	Examples 6, 7
Lock-free protocols	--timeout=60 or more	Example 8
Multi-file projects	v7 a.c b.c -I include -D CONFIG=1	pass the same flags as your build

16 When the result is UNKNOWN

UNKNOWN means neither a proof nor a bug was found in time. It is not a verdict on your program. Work through these steps in order — each is cheap to try.

1. **Give it more time.** `--timeout=120`, or 600 for hard protocols. Example 8 needs about 30 seconds; with 5 it stays open.
2. **Check only what matters.** Every check kind adds work. Example 10 has no heap memory: without the memory checks it is proven in seconds.
3. **Verify functions one at a time** with `--entry=f`. Smaller problems are decided faster, and a proof for every argument is stronger than one in context.
4. **State the real environment.** If a sensor only reads 0–4095, say so with `__VERIFIER_assume(0 <= raw && raw <= 4095);` — the certificate records it as an assumption.
5. **Add an intermediate assertion** at a key point, such as the invariant you expect at a loop head. It splits one hard proof into two easier ones.
6. **A longer budget:** `--timeout=300` for lock-free protocols and deep case analyses.
7. **Ask us.** Send the bundle to business.giolitlabs@gmail.com — hard cases are what we do for a living.

17 Continuous integration and the SDK

The exit code gates a build: 0 all proven, 1 a violation, 2 open checks, 3 a usage or compile error, 4 no valid license. SARIF output shows results as annotations in GitHub, GitLab and IDEs. A GitHub Actions step:

```
- name: Verify
  run: v7 --timeout=120 --checks=assertion,arithmetic,array-bounds \
      --sarif=verify.sarif --json=verify.json src/controller.c src/io.c -I include
- name: Upload results
  if: always()
  uses: github/codeql-action/upload-sarif@v3
  with: { sarif_file: verify.sarif }
```

Treat exit code 2 as a warning at first and tighten it as your code base becomes fully proven.

To gate a build from your own tool instead of the `v7` command, see the SDK (Section 18): `sdk-example/ci_gate.cpp` is such a gate in about 70 lines.

18 Embedding the verifier: the SDK

The verifier is a set of shared libraries with a small public C++ API. The `v7` command is one client of it; with the SDK you build others: a gate in your build system, a plug-in for your IDE or test framework, a tool that verifies every function of a module, or a service that issues certificates.

18.1 What the SDK contains

Part	What it is for
<code>include/giolit/License.hpp</code>	<code>acquireToken()</code> : a token for one run; <code>licenseStatus()</code> , <code>activateLicense()</code>
<code>include/giolit/Frontend.hpp</code>	<code>lowerProgram()</code> : C files to a verification task; <code>checkKinds()</code> , <code>validateChecks()</code>
<code>include/giolit/Backend.hpp</code>	<code>verifyProgram()</code> : every check proven, violated or open
<code>include/giolit/Report.hpp</code>	<code>writeReport()</code> : the PDF certificate or report and the result bundle
<code>include/vtask/</code>	the data the calls exchange: <code>Task</code> (checks, control-flow graph), <code>Result</code> , <code>CheckResult</code> , <code>Counterexample</code>
<code>lib/libgiolit_*.so</code>	one library per header above, and <code>libgiolit_interface</code> for the <code>vtask</code> types
<code>sdk-example/</code>	five example clients, the C programs they run on, and <code>build.sh</code>

Every client follows the same four steps:

1. `giolit::acquireToken()` — a token for one run of your trial or license. Every other entry point checks it and throws `giolit::AccessDenied` without a valid one.
2. `giolit::lowerProgram(token, files, options)` — the C program as a verification task: its control-flow graph and one *check* per operation that can fail (`task.checks`). `LowerOptions` take the check kinds, the entry function and compiler arguments (`-I`, `-D`), exactly like the `v7` options. A program that cannot be analysed throws `giolit::InputError`, with the line when it is known.
3. `giolit::verifyProgram(token, task, options, relower)` — the verification. `VerifyOptions::seconds` is the time budget. The `relower` callback translates the program again with fewer check kinds when the engine asks for it (a lighter model for the checks that do not concern memory); every example passes the same few lines.
4. Read `Verification::result.checks`, one `CheckResult` per check, in the order of `task.checks`: `status` (**PROVEN**, **VIOLATED**, **UNKNOWN**, **UNREACHABLE**), how it was decided, `cause` when it is open, and `cex`, the counterexample of a violation (its steps over the task's graph and a readable summary, `cex.text`).

18.2 Building a client

In the installed `sdk-example` folder, `./build.sh` builds all five examples; `./build.sh verify_file` builds one. For your own client, compile with C++17 against `include/` and link the libraries you call:

```
g++ -std=c++17 -O2 my_tool.cpp -I$PREFIX/include -L$PREFIX/lib -Wl,-rpath,$PREFIX/lib \
-lgiolit_license -lgiolit_frontend -lgiolit_backend -lgiolit_interface -o my_tool
```

Add `-lgiolit_report` when you call `writeReport`. `$PREFIX` is where `install.sh` put the verifier: `~/local/giolit-v7` by default, `/opt/giolit-v7` with `--system`.

18.3 Example: verifying a C program

`verify_file` is the smallest complete client. It runs on `programs/speed_limiter.c`, a vehicle speed limiter: every control cycle reads the driver's throttle request and the measured speed, and above the limit cuts the throttle back. The assertion is the controller's contract: the command is always a valid throttle value.

```
1  /* SDK demo program: a vehicle speed limiter.
2   *
3   * Every control cycle reads the driver's throttle request (per mille,
4   * 0..1000) and the measured speed (km/h). Above the limit the throttle is
5   * cut back by 50 per mille for every km/h of overspeed. The assertion is the
6   * controller's contract: the command is always a valid throttle value.
```

```

7  *
8  * ./verify_file programs/speed_limiter.c          -> all checks proven
9  * ./verify_file programs/speed_limiter.c -DEBUG   -> the assertion fails,
10 *      with the request and speed that make the command negative      */
11 extern int __VERIFIER_nondet_int(void);
12 extern _Bool __VERIFIER_nondet_bool(void);
13 extern void __VERIFIER_assume(int cond);
14 extern void reach_error(void);
15 #define ASSERT(c) do { if (!(c)) reach_error(); } while (0)
16
17 #define LIMIT 120          /* km/h */
18 #define MAX_THROTTLE 1000 /* per mille */
19
20 static int throttle_command(int request, int speed) {
21     if (request < 0) request = 0;
22     if (request > MAX_THROTTLE) request = MAX_THROTTLE;
23     if (speed <= LIMIT) return request;
24     int over = speed - LIMIT;
25 #ifdef BUG
26     return request - over * 50;          /* no saturation at zero */
27 #else
28     int cut = over >= 20 ? MAX_THROTTLE : over * 50;
29     return request > cut ? request - cut : 0;
30 #endif
31 }
32
33 int main(void) {
34     while (__VERIFIER_nondet_bool()) {          /* one control cycle */
35         int request = __VERIFIER_nondet_int();
36         int speed = __VERIFIER_nondet_int();
37         __VERIFIER_assume(speed >= 0 && speed <= 300);
38         int command = throttle_command(request, speed);
39         ASSERT(command >= 0 && command <= MAX_THROTTLE);
40     }
41     return 0;
42 }

```

The heart of `verify_file.cpp` is the four steps (option parsing and error handling left out; the folder has the whole program):

```

1  giolit::LowerOptions lower;          // how the program is translated
2  lower.checks = "all";                // every check kind
3  lower.compilerArgs = {"-DEBUG"};    // as on your compiler's command line
4  giolit::VerifyOptions verify;       // how it is verified
5  verify.seconds = 60;
6
7  // 1. a token: one run of your trial or license
8  giolit::TokenGrant grant = giolit::acquireToken();
9  if (!grant.granted) { std::cerr << grant.message << "\n"; return 4; }
10
11 // 2. the C program as a verification task
12 vtask::Task task = giolit::lowerProgram(grant.token, files, lower);
13
14 // 3. verify every check (relower: the same program, fewer check kinds)
15 auto relower = [&](const giolit::LowerVariant& v) {
16     giolit::LowerOptions o = lower;
17     if (!v.checks.empty()) o.checks = v.checks;
18     return giolit::lowerProgram(grant.token, files, o);
19 };
20 giolit::Verification v = giolit::verifyProgram(grant.token, task, verify, relower);
21
22 // 4. one result per check
23 for (const vtask::Check& c : task.checks) {
24     const vtask::CheckResult& r = v.result.checks.at(c.id);
25     std::cout << c.loc.file << ":" << c.loc.line << ": " << vtask::toString(r.status)
26         << " [" << c.kind << "]" << " " << c.message << "\n";
27     if (r.status == vtask::CheckStatus::Violated)
28         std::cout << " " << r.how << ": " << r.cex.text << "\n";
29 }

```

On the program as written, every check is proven:

```
$ ./verify_file programs/speed_limiter.c
programs/speed_limiter.c:24: PROVEN [overflow] int32 overflow in - (invariant (without the memory model))
programs/speed_limiter.c:28: PROVEN [overflow] int32 overflow in * (invariant (without the memory model))
programs/speed_limiter.c:29: PROVEN [overflow] int32 overflow in - (invariant (without the memory model))
programs/speed_limiter.c:39: PROVEN [assertion] assertion failure (reach_error reached) (invariant (without the memory model))
programs/speed_limiter.c:41: PROVEN [valid-memcleanup] allocated memory not freed at program exit (decided statically)
programs/speed_limiter.c:41: PROVEN [valid-memtrack] allocated memory may be lost (not freed at program exit) (decided statically)
6 checks: 0 violated, 0 unknown (0.06 s)
```

With `-DEBUG` the cut-back is not saturated at zero. The assertion is violated, and the counterexample gives the inputs in the order the program reads them: in the second cycle a request of `-2147483647` (clamped to 0) at a speed of 232 km/h makes the command negative:

```
$ ./verify_file programs/speed_limiter.c -DEBUG
...
programs/speed_limiter.c:39: VIOLATED [assertion] assertion failure (reach_error reached)
counterexample (concrete run): concrete run with inputs __VERIFIER_nondet_bool()=1,
__VERIFIER_nondet_int()=13, __VERIFIER_nondet_int()=2, __VERIFIER_nondet_bool()=1,
__VERIFIER_nondet_int()=-2147483647, __VERIFIER_nondet_int()=232:
reach_error reached at programs/speed_limiter.c:39 (114 steps)
...
6 checks: 1 violated, 0 unknown (0.05 s)
```

The exit code follows the `v7` convention (0 all proven, 1 a violation, 2 open checks), so `verify_file` already works as a build gate.

18.4 Example: verifying functions one by one

A library is verified best function by function. With `LowerOptions::entry` set to a function, its parameters take every value of their types, so a function proven safe is safe for every possible caller (the same as `v7 --entry`, Section 11). `unit_check` does this for each function named on its command line, on `programs/int_math.c`:

```
1  /* SDK demo program: integer helpers, each verified on its own.
2   *
3   * unit_check verifies every function as an entry point: its parameters take
4   * every value of their types, so a PROVEN function is safe for every
5   * caller, and a VIOLATED one comes with the arguments that break it.
6   *
7   * ./unit_check programs/int_math.c average_bad average_ratio_bad ratio */
8  #include <limits.h>
9
10 /* (a + b) overflows for large arguments of the same sign */
11 int average_bad(int a, int b) {
12     return (a + b) / 2;
13 }
14
15 /* halves first: the sum of two halves always fits */
16 int average(int a, int b) {
17     return a / 2 + b / 2 + (a % 2 + b % 2) / 2;
18 }
19
20 /* b == 0 divides by zero; INT_MIN / -1 overflows */
21 int ratio_bad(int a, int b) {
22     return a / b;
23 }
24
25 int ratio(int a, int b) {
26     if (b == 0) return 0;
27     if (a == INT_MIN && b == -1) return INT_MAX;
28     return a / b;
29 }
```

Its loop: a token and a verification per function, then the arguments of each counterexample. They are the values of the parameters at the function's entry, read from the counterexample's steps:

```
1  for (const std::string& function : functions) {
```

```

2   giolit::TokenGrant grant = giolit::acquireToken(); // one run per function
3   giolit::LowerOptions o = lower;
4   o.entry = function; // parameters: arbitrary inputs
5   vtask::Task task = giolit::lowerProgram(grant.token, files, o);
6   giolit::Verification v = giolit::verifyProgram(grant.token, task, verify, relower);
7   for (const vtask::Check& c : task.checks) {
8       const vtask::CheckResult& r = v.result.checks.at(c.id);
9       if (r.status == vtask::CheckStatus::Violated)
10          std::cout << "   line " << c.loc.line << ": " << c.message << "\n   when called with "
11              << arguments(v.task, r.cex, function) << "\n";
12   }
13 }
14
15 // the parameters' values at the function's entry: the named slots of its entry node
16 std::string arguments(const vtask::Task& task, const vtask::Counterexample& cex,
17     const std::string& function) {
18     for (const vtask::CexStep& step : cex.steps) {
19         const vtask::Node& n = task.nodes[step.node];
20         if (n.name != function + "::entry") continue;
21         std::string out;
22         for (size_t i = 0; i < n.slots.size(); ++i)
23             if (n.slots[i].name[0] != '@' && n.slots[i].name[0] != '%') // not internal
24                 out += (out.empty() ? "" : ", ") + n.slots[i].name + " = " + std::to_string(step.values[i]);
25         return out;
26     }
27     return "";
28 }

```

Two of the four functions are unsafe, each with the arguments that break it:

```

$ ./unit_check programs/int_math.c average_bad average ratio_bad ratio
average_bad  UNSAFE  9 of 10 checks proven
   line 12: int32 overflow in +
   when called with a = -2147483648, b = -2147483648
average      SAFE   10 of 10 checks proven
ratio_bad    UNSAFE  8 of 10 checks proven
   line 22: division or remainder by zero
   when called with a = -2147483648, b = 0
   line 22: int32 overflow in / or % (INT_MIN / -1)
   when called with a = -2147483648, b = -1
ratio        SAFE   10 of 10 checks proven

```

average and ratio, the corrected versions, are proven for all 2^{64} argument pairs: no test suite could cover that.

18.5 The other examples

Example	What it shows	Try
certify	verification plus writeReport: the PDF certificate, counterexample or verification report and the bundle (JSON, SARIF, proof, reproduce.sh)	<code>./certify --issued-to="ACME" out programs/speed_limiter.c</code>
license_info	the license state and the check kinds without spending a run; validating a <code>--checks</code> option	<code>./license_info --checks=assertion,arithmetic</code>
ci_gate	a build gate over several files, one line per check	<code>./ci_gate --checks=assertion my_module.c</code>

 **Good practice with the SDK** Check `TokenGrant::granted` and show its message: it says why no run is available and how to get a license. Catch `giolit::InputError` for programs that do not compile. Give the front end the same budget as the verification (`lower.seconds = verify.seconds`): its concrete runs take a share of it. Each token is one run; acquire one per program or entry point you verify, as `unit_check` does.

19 Limits and good practice

-
- **The result is about the analysed code and its assumptions.** Functions without a body are treated as arbitrary inputs, not as their real implementation. Include the code you want covered, or model a library function with a small stub.
 - **Keep verification harnesses** next to the code and run them in CI: they are executable specifications.
 - **Prefer small entry points.** Verifying functions one at a time scales better than a whole program at once.
 - **Floating point** and some bit manipulations are over-approximated: they can cause **UNKNOWN**, never a false **PROVEN**.

A Command reference

Option	Meaning (default)
--checks=LIST	check kinds, comma-separated, or a group (all)
--entry=NAME	the function to verify from; its parameters are arbitrary (main)
--timeout=SECONDS	the total time budget (60)
--report=DIR	write the bundle: PDF, JSON, SARIF, proof
--issued-to=NAME	the recipient named on the PDF
--json=FILE, --sarif=FILE, --proof=FILE	machine-readable results, SARIF 2.1.0, the full proof
--verbose	complete counterexample traces
-I DIR, -D NAME=V, -std=...	passed to the C compiler, as in your build
--license	the license: runs and days left (uses no run)
--activate=FILE	apply a purchased license (asks for name, address, password)
--benchmark	one SV-COMP style verdict (SAFE / UNSAFE / UNKNOWN)

Exit codes: 0 all proven, 1 a violation, 2 open checks, 3 usage or input error, 4 no valid license.

B Trial and professional license

Trial. 1000 runs within 50 days of the first run, on one computer; documents are marked “Trial version”. `v7 --license` shows what is left.

Professional license. Your license comes as a file and a letter with your license number and password. Activate it once:

```
$ v7 --activate acme.lic
Buyer name: Acme Embedded Pvt. Ltd.
Buyer address: 12 Main Street, Kochi 682001, India
License password: *****
Activated: professional license GIOLIT-PRO-2026-000001, licensed to Acme Embedded Pvt. Ltd.
```

From then on, every document carries your license number and company name at the top, without trial marks, and each PDF is protected with its own password, written to `certificate-password.txt` in the same folder. The license covers the runs and the period stated in your letter. For licenses, renewals and volume pricing: **business.giolit-labs@gmail.com**.

C Glossary

Check

a place where the program could fail (an assertion, an addition that could overflow, a pointer access), verified separately.

Counterexample

a concrete input and execution that violates a check.

Invariant, annotation

a fact that holds every time execution reaches a program point.

Inductive

an invariant that holds initially and is preserved by every step — which makes it a proof.

Contract

a function’s precondition (*requires*) and postcondition (*ensures*).

Owicki-Gries

the proof method for concurrent programs: per-thread proofs plus interference freedom.

Over-approximation

a model with at least all behaviours of the program: proofs on it are sound.

Entry function

where verification starts (main, or `--entry`).

D Benchmark results

The package's `results/` folder holds the verifier's result — console output, PDF, JSON, SARIF and proof — for every benchmark in `benchmarks/`, made with this version on a 4-core laptop. Every one is a correct SAFE (all checks proven) or UNSAFE (a real violation found) result.

Category	Benchmarks	Proven safe	Bug found	Median time
Sequential (SV-COMP)	47	33	14	10 s
Recursive (SV-COMP)	92	48	44	10 s
Concurrent (SV-COMP and thread suite)	24	12	12	12 s
Tutorial examples	14	9	5	8 s
Total	177	102	75	

Median time per benchmark includes writing the PDF (about 8 s); 4-core laptop, 60-second budget.



Giolit — Formal Verification Services

Mathematical proof that your software is free of run-time errors.

We offer formal verification as a service: proofs of absence of run-time errors (overflow, division by zero, out-of-bounds accesses, invalid pointers, memory leaks) and of functional properties for C code, certificates for safety standards (DO-178C, ISO 26262, IEC 61508), review of counterexamples with your engineers, and training for your team.

 www.giolit.com

 business.giolitlabs@gmail.com

